International Institute of Information Technology, Bangalore

Software Production Engineering Mini Project Report

Scientific Calculator

Submitted by

Piyush Singh (MT2025088)

Under the guidance of

Prof. B. Thangaraju

Abstract

This project implements a Scientific Calculator integrated with DevOps practices. The calculator performs several mathematical operations including square root, factorial, natural logarithm, and power.

The main objective of this project is to demonstrate how a simple application can be integrated with a complete DevOps workflow. The system uses GitHub for version control, Jenkins for CI/CD automation, Docker for containerisation, DockerHub for container image storage, and Ansible for automated deployment. This automated workflow improves the reliability and efficiency of software development and deployment.

Advantages of DevOps:

1. Faster Software Delivery
DevOps enables faster development and deployment through automation and continuous integration and continuous deployment (CI/CD), reducing the time it takes to deliver working software to users significantly.

2. Improved Software Quality
Automated testing and continuous monitoring help identify bugs early in the development process, resulting in more reliable and high-quality software that fails less often in production.

3. Better Collaboration
DevOps promotes strong collaboration between development and operations teams, reducing communication gaps and improving overall team productivity through shared goals and tooling.

4. Increased Efficiency through Automation
DevOps automates repetitive tasks such as testing, integration, and deployment, which reduces manual effort and minimises human errors throughout the software delivery lifecycle.

5. Enhanced Customer Satisfaction
Faster updates, fewer bugs, and improved performance lead to a better user experience and higher customer satisfaction as software reaches users more quickly and reliably.

Tools and Technologies Used

The following tools and technologies were used in the development and deployment of this project.

1. Programming Language – Python

Python was used to develop the scientific calculator application and implement the mathematical operations including square root, factorial, natural logarithm, and power. The built-in unit test framework was used to write automated tests for each operation.

2. Version Control – Git and GitHub

Git was used to track changes in the project source code, while GitHub served as the remote repository for storing and managing the project files. A webhook was configured to automatically trigger Jenkins on every push to the main branch.

3. Continuous Integration – Jenkins

Jenkins was used to automate the CI/CD pipeline. It automatically builds the project, runs tests, creates Docker images, and triggers deployment using Ansible whenever changes are pushed to the GitHub repository.

4. Containerisation – Docker

Docker was used to package the application and its dependencies into a container, ensuring consistent execution across different environments. The image is built from a Dockerfile using the official Python slim base image to keep the size minimal.

5. Container Registry – DockerHub

DockerHub was used to store Docker images generated during the CI/CD pipeline. The image is publicly available at hub.docker.com/r/kabubabu/scientific-calculator and is pulled by Ansible during the automated deployment stage.

6. Deployment Automation – Ansible

Ansible was used to automate the deployment process by pulling the Docker image from DockerHub, stopping the existing container, and running a new container. The playbook is idempotent and can be safely run multiple times without side effects.

STEP 1: Source Code Management

Source Code Management is used to maintain version history and manage changes in the project. Without version control it is impossible to track what changed, who changed it, and why — making collaboration and rollback extremely difficult.

In this project, Git was used as the version control system while GitHub was used as the remote repository. The repository contains the application source file, test file, Docker configuration, Jenkins pipeline file, and Ansible deployment files.

Whenever code changes are made locally, they are committed and pushed to GitHub. This repository acts as the central source of truth for triggering the CI/CD pipeline. A webhook configured on the repository notifies Jenkins automatically on every push to main.

GitHub Repository:

https://github.com/Coder-Galaxy/scientific-calculator

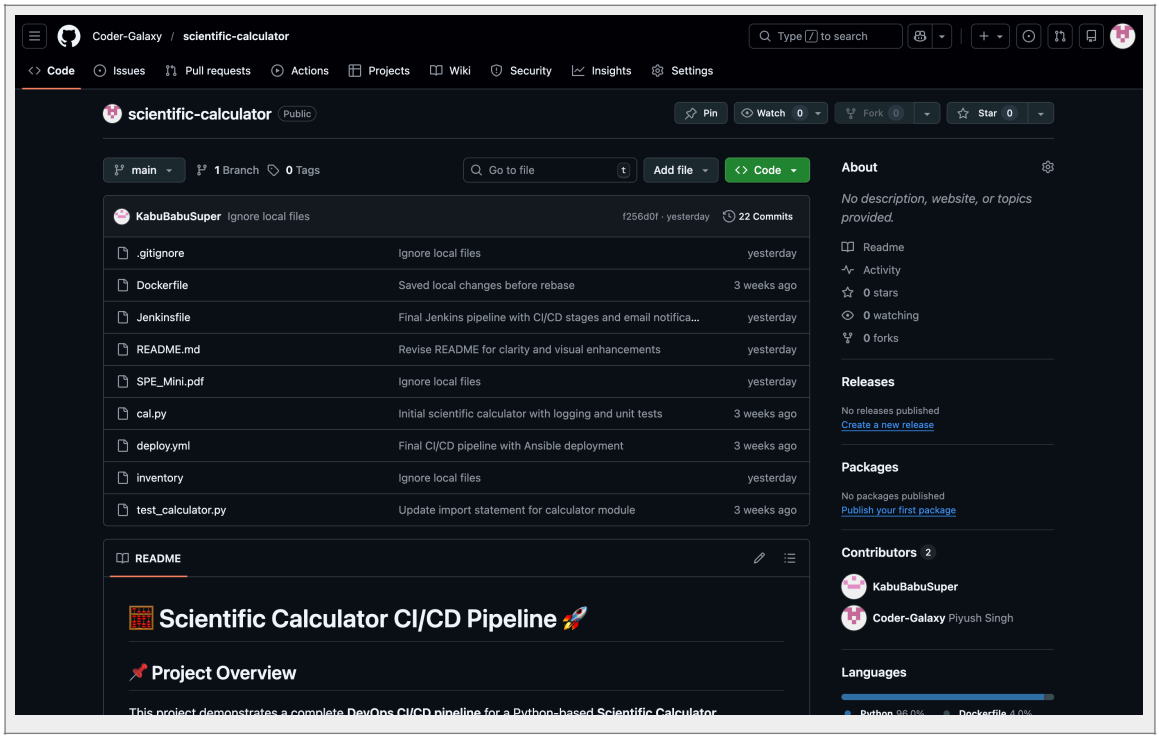


Figure 1: GitHub Repository Structure

STEP 2: Continuous Integration using Jenkins

Jenkins is an open-source automation server used for Continuous Integration and Continuous Deployment. It monitors the GitHub repository for changes and automatically executes the defined pipeline whenever a push event is received through the webhook.

In this project, Jenkins was configured to automatically trigger the pipeline whenever changes are pushed. The pipeline performs several automated tasks: checking out the repository, installing dependencies, running automated tests, building the Docker image, pushing the image to DockerHub, and deploying the application using Ansible.

Jenkins Pipeline Setup:

The pipeline was configured as a Pipeline job sourced from the Jenkinsfile in GitHub. DockerHub credentials were securely added to Jenkins Credential Manager under the ID dockerhub-creds. The SMTP email server was configured to send build notifications.

Pipeline Stages Explained:

1. Declarative: Checkout SCM: Jenkins automatically fetches the latest code from GitHub at the start of every build. This ensures the pipeline always works on the exact commit that triggered the build.

2. Install Dependencies: Jenkins installs required Python packages using pip. The stage proceeds safely even when no external packages are required beyond Python's standard library.

3. Run Tests: All six unit tests run via Python's unittest runner. If any test fails, the build is marked FAILED and all remaining stages are skipped — acting as a strict quality gate.

4. Docker Login: Jenkins securely retrieves the DockerHub credentials from the Credential Manager and authenticates using the password-via-stdin method, keeping the token out of build logs.

5. Build Docker Image: Jenkins builds the Docker image from the Dockerfile, tagging it with the DockerHub repository name and the latest tag using the slim Python base image.

6. Push Docker Image: The tested image is pushed to hub.docker.com/r/kabubabu/scientific-calculator. After this stage the latest tested version is available for Ansible to deploy.

7. Deploy with Ansible: Jenkins invokes the Ansible playbook. Ansible pulls the latest image, stops the old container if running, and starts a fresh container — completing automated deployment.

8. Declarative: Post Actions: A success email is sent on passing builds and a failure alert on any stage failure. Jenkins always logs out of DockerHub to clean up the session securely.

Figure 2: Jenkins Pipeline Stage View

STEP 3: Docker Containerisation

Docker is used to package the application and its dependencies into a container, ensuring the application runs consistently across different environments.

In this project, Jenkins builds the Docker image automatically during the CI/CD pipeline and pushes the image to DockerHub. The image is based on the official Python slim base image and sets the calculator as the container startup process.

DockerHub Repository:

https://hub.docker.com/r/kabubabu/scientific-calculator

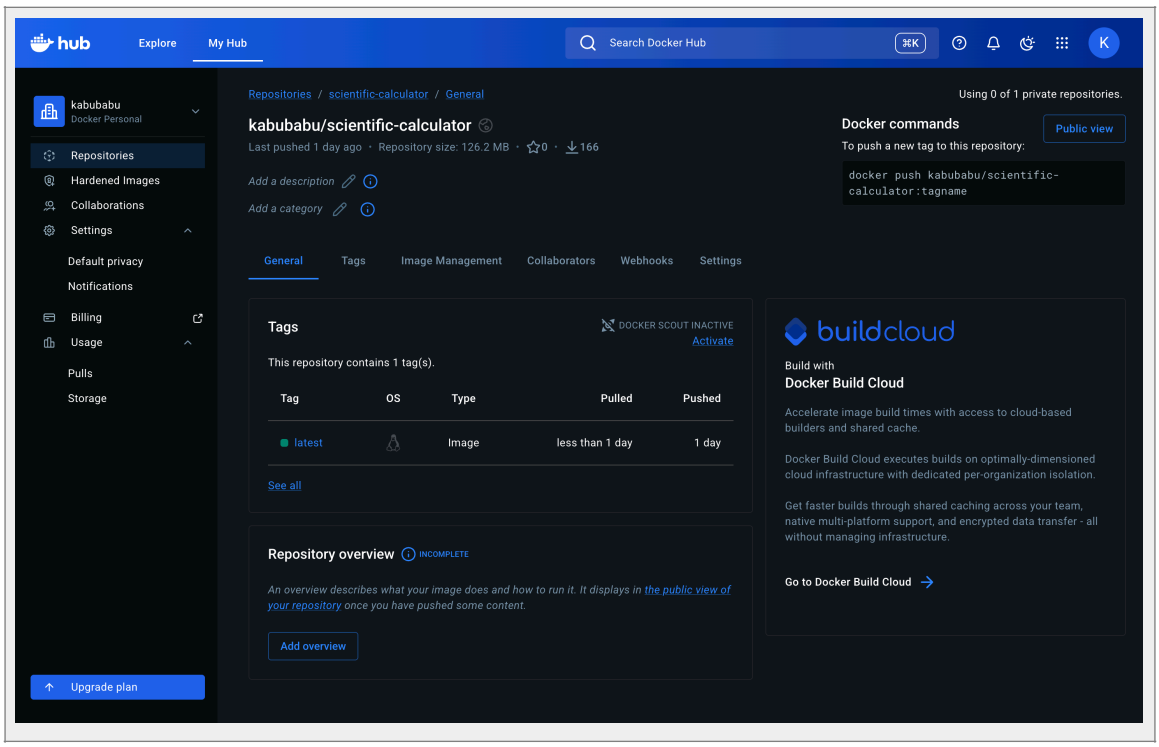
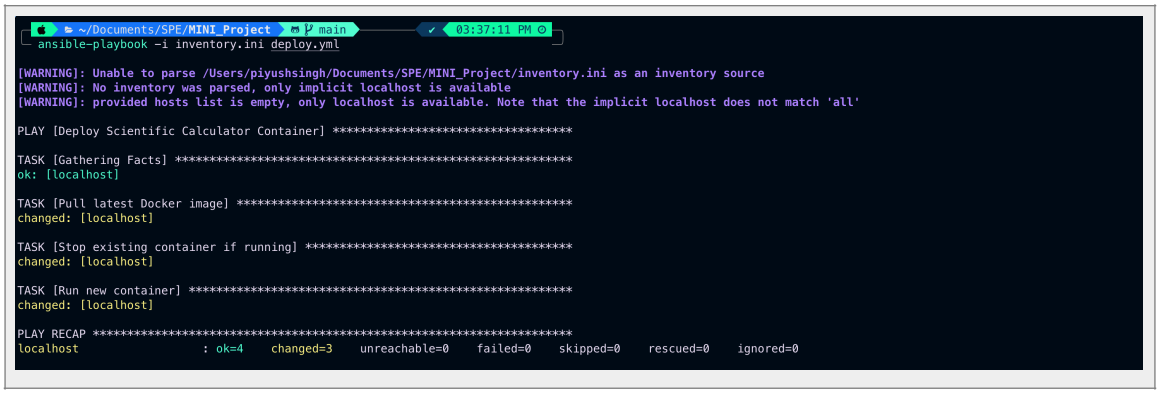


Figure 3: Docker Hub Repository Page

STEP 4: Deployment using Ansible

Ansible is an automation tool used for configuration management and deployment. In this project, Ansible was used to deploy the Docker container automatically after the Jenkins pipeline completes. The playbook pulls the latest image from DockerHub, stops any existing container, and starts a new one. This automated deployment ensures consistent and reliable application delivery.



```
~/Documents/SPE/MINI_Project  main                                  ✔  03:37:11 PM
ansible-playbook -i inventory.ini deploy.yml

[WARNING]: Unable to parse /Users/piyushsingh/Documents/SPE/MINI_Project/inventory.ini as an inventory source
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [Deploy Scientific Calculator Container] ***********************************

TASK [Gathering Facts] *********************************************************
ok: [localhost]

TASK [Pull latest Docker image] ************************************************
changed: [localhost]

TASK [Stop existing container if running] **************************************
changed: [localhost]

TASK [Run new container] *******************************************************
changed: [localhost]

PLAY RECAP *********************************************************************
localhost                  : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 4: Deployment of Calculator Container using Ansible

Automated Post-Build Notification (Email)

The Jenkins pipeline was configured to send an email notification after every pipeline execution. This notifies the developer whether the build and deployment completed successfully or failed. The Email Extension Plugin integrates Jenkins with an SMTP server. Once the pipeline finishes, Jenkins automatically sends an email containing the build status and relevant details.

Key Configuration Parameters:

- SMTP Server: smtp.gmail.com
- SMTP Port: 587 (TLS)
- Credentials: Jenkins Credentials Provider using a Google App Password
- Trigger Condition: Email sent after pipeline success or failure

SUCCESS: Jenkins Build #22 Inbox ×

address not configured yet <piyushsinghcu@gmail.com> Tue, Mar 7, 3:37 PM (1 day ago)
to me

Build SUCCESSFUL!

Project: Scientific Calculator CI/CD Pipeline
Job Name: scientific-calculator
Build Number: 22

Docker image built and deployed successfully.

Figure 5: Receiving build status on mail

Final Run and Conclusion

The Scientific Calculator application runs successfully inside a Docker container after the CI/CD pipeline execution. The Jenkins pipeline automates the entire workflow including code integration, testing, containerisation, and deployment.

By integrating tools such as GitHub, Jenkins, Docker, DockerHub, and Ansible, the project demonstrates a complete DevOps pipeline. This highlights how DevOps practices improve development efficiency, reduce manual effort, and ensure reliable software deployment.

```
Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log (ln)
4. Power (x^y)
5. Exit
Enter your choice: 1
Enter number: 2
Result: 1.4142135623730951
```

```
Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log (ln)
4. Power (x^y)
5. Exit
Enter your choice: 2
Enter number: 3
Result: 6
```

```
Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log (ln)
4. Power (x^y)
5. Exit
Enter your choice: 3
Enter number: 4
Result: 1.3862943611198906
```

```
Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log (ln)
4. Power (x^y)
5. Exit
Enter your choice: 4
Enter base: 4
Enter exponent: 2
Result: 16.0
```

Figure 6: Scientific Calculator Running Inside Docker Container

GitHub Repository:

https://github.com/Coder-Galaxy/scientific-calculator

DockerHub Repository:

https://hub.docker.com/r/kabubabu/scientific-calculator